

LAB-1

1. Design a webpage that uses different heading tags (<h1> to <h6>) to display titles and subtitles. Write a paragraph using the <p> tag and format important text using bold (or) and emphasis (<i> or) tags. Add HTML comments in the code to explain some part of the code. Use escape characters in order to explain about each tag.
2. Create a webpage that includes at least two hyperlinks using the <a> tag. One link should open in the same tab and the other should open in a new tab.
3. Create a student registration form using HTML only that collects user information such as full name, email, password, age, gender, course and acceptance of terms and conditions. Use appropriate form elements including text fields, radio buttons, dropdown lists and checkboxes. Apply basic HTML validation using attributes such as required, min, max and pattern. As a challenge, prevent the form from being submitted if the user's age is less than 16 and display an alert message saying "Registration Successful" when the form is submitted successfully.
4. Create a personal profile card using HTML only that displays a user's basic information in a structured format. The profile card should include a profile image, full name, short description or bio, research interest, skills and contact information such as email or website using hyperlinks.
5. Create a webpage using HTML only that displays student information in a tabular format. Use the <table>, <tr>, <th>, and <td> tags to design the table with appropriate headings and rows. Include at least three rows of data and use attributes such as

border, cellpadding, and cellspacing to improve table readability. Ensure the table is properly structured and easy to understand.

More Practice Questions:

6. Create a multi-section webpage using semantic HTML elements only. The webpage should include a `<header>` section containing the title of the website, a `<nav>` section with navigation links, multiple `<section>` elements for different content areas, and a `<footer>` section for footer information. Each navigation link should use anchor tags (`<a>`) to scroll to its corresponding section using section id attributes. Ensure proper use of semantic tags, meaningful section headings, and well-structured HTML code. Navigation between sections should work correctly when the links are clicked.
7. Create a webpage using HTML only that demonstrates the use of different types of lists. Display an unordered list showing at least five favorite subjects or skills and an ordered list showing three daily activities or steps of a process. Also include a description list (`<dl>`) with at least two terms and their corresponding descriptions. Demonstrate list nesting by placing one list inside another and as a mini challenge, change the list style using the `type` attribute and create a mixed list where an unordered list is placed inside an ordered list.

LAB-2

1. Create a webpage that demonstrates the CSS Box Model. Design multiple `<div>` elements and apply margin, padding, border, width, and height properties to each element. Use different border styles and border radius values. Show the difference between box-sizing: content-box and box-sizing: border-box by applying them to different elements. Ensure proper spacing between elements so that the effect of each box model property is clearly visible.
2. Design a webpage using CSS that demonstrates basic text styling and formatting. Apply different text colors, background colors, font families, font sizes, font styles, font weights and text alignment properties to headings and paragraphs.
3. Create a webpage that demonstrates CSS positioning and display properties. Use different elements and apply static, relative, absolute, fixed, and sticky positioning. Apply display values such as block, inline, inline-block, and none. Use z-index to show overlapping elements and apply overflow properties such as hidden, scroll and auto where necessary.
4. Design a responsive webpage layout using CSS Flexbox. Create a container with multiple items and apply flexbox properties such as flex-direction, justify-content, align-items and flex-wrap. Demonstrate the use of flex-grow, flex-shrink and flex-basis on flex items. Use flexbox to design a navigation bar and a card-style layout that adjusts properly on different screen sizes.
5. Design a webpage layout using CSS Grid. Create a grid-based structure that includes a header, sidebar, main content area, and footer. Use grid-template-rows, grid-template-columns, gap, grid-row, and grid-column properties to arrange the layout. Ensure the grid layout is clean, well-structured, and responsive.

6. Create a webpage that demonstrates advanced CSS features. Use pseudo-classes such as hover, focus, and active, and pseudo-elements such as before and after. Apply CSS transitions, transforms, and animations using keyframes. Use CSS variables to define common colors and apply media queries to make the webpage responsive for different screen sizes.
7. Design a student registration form using HTML and CSS that collects user information such as username, email, password, age, gender, course and acceptance of terms and conditions. Use CSS to style the form elements including text fields, radio buttons, dropdown lists and checkboxes to make the form visually appealing and user-friendly. Apply layout techniques such as spacing, alignment, borders, background colors and font styling to organize the form in a structured manner. Ensure that form validation messages and required fields are clearly visible and highlight the input fields when focused. As a challenge, use CSS to display a message box or style the alert area that appears if the user's age is less than 16 and apply hover or focus effects on the submit button to indicate interactivity.
8. Create a personal profile card using HTML and CSS that displays a user's basic information in a visually structured format. The profile card should include a profile image, full name, short description or bio, research interests, skills and contact information such as email or website using hyperlinks. Use CSS to style the card with appropriate borders, shadows, spacing, background colors and font styles. Apply layout techniques such as Flexbox or Grid to align the content inside the card. Include hover effects, transitions or subtle animations to enhance the presentation of the card and ensure that all information is clearly readable and well-organized.
9. Create a webpage using HTML and CSS that displays student information in a tabular format. Use the `<table>`, `<tr>`, `<th>`, and `<td>` tags to design the table with appropriate headings and at least seven rows of student data. Apply CSS to style the table

including borders, padding, spacing, text alignment, background colors and font styling. Use alternate row coloring, hover effects, and table caption styling to improve readability and visual appeal. Ensure that the table is responsive and maintains proper structure and alignment on different screen sizes.

LAB-3 JavaScript

JS basics Practice:

1. Write an arrow function to calculate the sum of all elements in an array.
2. Write a program to calculate the factorial of a number using:
 - a. for loop
 - b. while loop
3. Write a function to check if a string is a palindrome.
4. Show the difference between var, let and const inside a loop.
5. Create an arrow function that returns an object and use template literals to print: Hello, my name is ____ and I am ____ years old.
6. Create a multi-line HTML template using template literals and insert it into the DOM.
7. Write a function using rest parameters that accepts any number of arguments and returns their product.
8. Copy an object using the spread operator and modify one property.
9. Convert an array of strings to uppercase using map().
10. Remove duplicate elements from an array.
11. Sort an array of objects based on age or marks.
12. Loop through object properties using for...in.

For Lab:

1. Create a site that needs to have two buttons; one for adding an item into the list and another for removing the last item from the list.

Dynamic List Controller

Add ItemRemove Last Item

Type an item to insert into List:

- Apple
- Ball

2. Create a registration form with the following fields: username, email, role (dropdown: Student or Teacher), password, confirm password, full name, email, password, age, gender, course and acceptance of terms and conditions. Validate the form according to these rules:
 - a. Email must match the role:
 - i. If Student → email must end with [@student.edu.np](mailto:student.edu.np)
 - ii. If Teacher → email must end with [@tutor.edu.np](mailto:tutor.edu.np)
 - b. Confirm Password must match Password.Highlight invalid inputs with a red border and show error messages. Prevent form submission if any validation fails.

3. Build a Dynamic Table that adds and deletes rows.

Dynamic Table

Name	Age	Add Row
-----------------------------------	----------------------------------	--

Name	Age	Actions
Ram	31	Delete

4. Design and develop a simple interactive webpage that displays a light bulb image along with a control button. When the user clicks the button, the light bulb should toggle between ON and OFF states by changing its image dynamically. At the same time, a status message on the page should update to clearly indicate whether the bulb is currently ON or OFF. Additionally, the button text should change accordingly (for example, "Turn ON" when the bulb is OFF and "Turn OFF" when the bulb is ON). Implement the functionality using JavaScript to handle the click event and manipulate the DOM elements dynamically.

Light Bulb Toggle



Turn ON

The bulb is OFF

5. Design and develop a simple webpage that contains a password input field along with a “Show / Hide” toggle button. When the user clicks the button, the password should dynamically switch between hidden (password type) and visible (text type). At the same time, the button text should update accordingly (e.g., display “Show” when the password is hidden and “Hide” when it is visible).

Password Show / Hide Toggle



Password is Hidden

Practice (no need to submit this)

6. Fetch user data from an API and display it in card format.

LAB-4 React

1. Design and develop a simple React component that displays a light bulb image along with a control button. When the user clicks the button, the bulb should toggle between ON and OFF states by changing the image dynamically. At the same time, a status message should update to indicate whether the bulb is currently ON or OFF. The button text should also change accordingly (for example, “Turn ON” or “Turn OFF”). Implement the functionality using React state and event handlers.
2. Design and develop a React component that contains a password input field along with a “Show / Hide” toggle button. When the user clicks the button, the password field should switch dynamically

between hidden and visible states. The button text should also update accordingly. Use React state to manage the visibility of the password.

3. Design a React application where a parent component maintains a list of items in its state and passes the list along with add and delete functions to a child component using props. The child component should display the list and allow users to modify it through the provided buttons.
4. Create a React application with multiple pages such as Home, About, and Contact. Implement navigation between these pages using React Router so that page transitions occur without reloading the browser. Display a navigation menu that allows users to switch between pages seamlessly.
5. Create a React application that uses state to maintain a counter value. Provide buttons to increment, decrement, and reset the counter. Display the updated counter value dynamically on the screen.